

BLACK BOX TESTING

Black box testing is a technique of software testing which examines the functionality of software without peering into its internal structure or coding. The primary source of black box testing is a specification of requirements that is stated by the customer.

- In this method, tester selects a function and gives input value to examine its functionality.
- Checks whether the function is giving expected output or not.
- If the function produces correct output, then it is passed in testing, otherwise failed.
- The test team reports the result to the development team and then tests the next function.
- After completing testing of all functions if there are severe problems, then it is given back to the development team for correction.



Steps of black box testing

- The black box test is based on the specification of requirements, so it is examined in the beginning.
- In the second step, the tester creates a positive test scenario and an adverse test scenario by selecting valid and invalid input values to check that the software is processing them correctly or incorrectly.
- In the third step, the tester develops various test cases such as decision table, all pairs test, equivalent division, error estimation, cause-effect graph, etc.
- The fourth phase includes the execution of all test cases.
- In the fifth step, the tester compares the expected output against the actual output.
- In the sixth and final step, if there is any flaw in the software, then it is cured and tested again.

Test procedure

The test procedure of black box testing is a kind of process in which the tester has specific knowledge about the software's work, and it develops test cases to check the accuracy of the software's functionality.

- It does not require programming knowledge of the software. All test cases are designed by considering the input and output of a particular function.
- A tester knows about the definite output of a particular input, but not about how the result is arising.
- There are various techniques used in black box testing for testing like decision table technique, boundary value analysis technique, state transition, All-pair testing, cause-effect graph technique, equivalence partitioning technique, error guessing technique and use case technique.

Test cases

- Test cases are created considering the specification of the requirements.
- These test cases are generally created from working descriptions of the software including requirements, design parameters, and other specifications.
- The test designer selects both positive test scenario by taking valid input values and adverse test scenario by taking invalid input values to determine the correct output.
- Test cases are mainly designed for functional testing but can also be used for non-functional testing.
- Test cases are designed by the testing team, there is not any involvement of the development team of software.

Techniques Used in Black Box Testing

1. Decision table technique
2. Boundary value technique

3. State transition technique
4. All pair testing technique
5. Cause-Effect technique
6. Equivalence partitioning technique
7. Error guessing technique
8. Use case technique

1. Decision table technique

Decision table technique is one of the widely used case design techniques for black box testing. This is a systematic approach where various input combinations and their respective system behavior are captured in a tabular form.

This technique is used to pick the test cases in a systematic manner; it saves the testing time and gives good coverage to the testing area of the software application.

This technique is related to the correct combination of inputs and determines the result of various combinations of input. To design the test cases by decision table technique, we need to consider conditions as input and actions as output.

Example: Email account

To use an email account, for this you need to enter the email and its associated password. If both email and password are correctly matched, the user will be directed to the email account's homepage; otherwise, it will come back to the login page with an error message specified with "Incorrect Email" or "Incorrect Password."

Email (condition1)	T	T	F	F
Password (condition2)	T	F	T	F
Expected Result (Action)	Account Page	Incorrect password	Incorrect email	Incorrect email

- In this table, there are four conditions or test cases to test the login function. In the first condition if both email and password are correct, then the user should be directed to account's Homepage.
- In the second condition if the email is correct, but the password is incorrect then the function should display Incorrect Password. In the third condition if the email is incorrect, but the password is correct, then it should display Incorrect Email.
- In fourth and last condition both email and password are incorrect then the function should display Incorrect Email.
- The testing team also includes all possible test cases so that upcoming bugs can be cured at testing level.

Number of possible conditions = $2^{\text{Number of Values of the second condition}}$

Number of possible conditions = $2^2 = 4$

- While using the decision table technique, a tester determines the expected output, if the function produces expected output, then it is passed in testing, and if not then it is failed. Failed software is sent back to the development team to fix the defect.

2. Boundary value technique

- Boundary value analysis is one of the widely used case design technique for black box testing. It is used to test boundary values.
- The tester focuses on, while entering boundary value whether the software is producing correct output or not.
- Boundary values are those that contain the upper and lower limit of a variable.
- Assume that, age is a variable of any function, and its minimum value is 18 and the maximum value is 30, both 18 and 30 will be considered as boundary values.
- Test cases are developed for each and every value of the range.
- Testing of boundary values is done by making valid and invalid partitions.
- There is a function that accepts a number between 18 to 30, where 18 is the minimum and 30 is the maximum value of valid partition, the other values of this partition are 19, 20, 21, 22, 23, 24, 25, 26, 27, 28 and 29. The invalid partition consists of the numbers which are less than 18 such as 12, 14, 15, 16 and 17, and more than 30 such as 31, 32, 34, 36 and 40. Tester develops test cases for both valid and invalid partitions to capture the behavior of the system on different input conditions.



Invalid test cases	Valid test cases	Invalid test cases
11, 13, 14, 15, 16, 17	18, 19, 24, 27, 28, 30	31, 32, 36, 37, 38, 39

- The software system will be passed in the test if it accepts a valid number and gives the desired output, if it is not, then it is unsuccessful.
- In another scenario, the software system should not accept invalid numbers, and if the entered number is invalid, then it should display error message.

3. State transition technique

- It is used to capture the behavior of the software application when different input values are given to the same function.

For example: ATMs

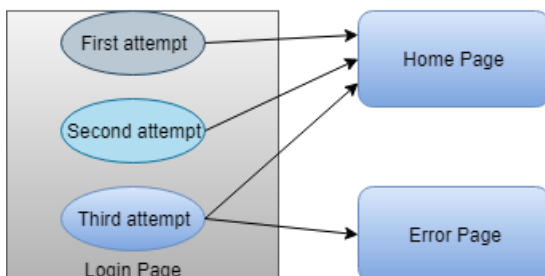
- When we withdraw money from it, it displays account details at last. Now we again do another transaction, then it again displays account details, but the details displayed after the second transaction are different from the first transaction, but both details are displayed by using the same function of the ATM. So the same function was used here but each time the output was different, this is called state transition.
- The login function we use email and password, it gives a specific number of attempts to access the application, after crossing the maximum number of attempts it gets locked with an error message.

Login Here

Email

Password

There is a login function of an application which provides a maximum three number of attempts, and after exceeding three attempts, it will be directed to an error page.



State transition table

State	Login	Validation	Redirected
S 1	First Attempt	Invalid	S2
S 2	Second Attempt	Invalid	S3

S 3	Third Attempt	Invalid	S5
S 4	Home Page		
S 5	Error Page		

In the above state transition table, we see that state S1 denotes first login attempt. When the first attempt is invalid, the user will be directed to the second attempt (state S2). If the second attempt is also invalid, then the user will be directed to the third attempt (state S3). Now if the third and last attempt is invalid, then the user will be directed to the error page (state S5).

But if the third attempt is valid, then it will be directed to the homepage (state S4).

Let's see state transition table if third attempt is valid:

State	Login	Validation	Redirected
S 1	First Attempt	Invalid	S2
S 2	Second Attempt	Invalid	S3
S 3	Third Attempt	Valid	S5
S 4	Home Page		
S 5	Error Page		

Using the above state transition table we can perform testing of any software application.

4. All pair testing technique

- All-pairs testing technique is also known as pairwise testing. It is used to test all the possible discrete combinations of values.
- This combinational method is used for testing the application that uses checkbox input, radio button input, list box, text box, etc.
- A function of a software application for testing, in which there are 10 fields to input the data, so the total number of discrete combinations is 10^4 (100 billion), but testing of all combinations is complicated because it will take a lot of time.
- Assume that there is a function with a list box that contains 10 elements, text box that can accept 1 to 100 characters, radio button, checkbox and OK button.

The input values are given below that can be accepted by the fields of the given function.

1. Check Box - Checked or Unchecked
2. List Box - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
3. Radio Button - On or Off
4. Text Box - Number of alphabets between 1 to 100.
5. OK - Does not accept any value, only redirects to the next page.

Calculation of all the possible combinations:

Check Box = 2
List Box = 10
Radio Button = 2
Text Box = 100
Total number of test cases = $2 \times 10 \times 2 \times 100$
= 4000

The total number of test cases, including negative test cases, is 4000.

- Testing of 4000 positive and negative test cases, is a very long and time-consuming process. Therefore, the task of the testing team is to reduce the number of test cases.
- The testing team considers the list box values in such a way that the first value is 0, and the other value can be any numeric number, neither positive nor negative. Ten values are now converted into 2 values.
- Values of checkbox and radio button cannot be reduced because each has a combination of only 2 values.
- At last, the value of the text box is divided into three input categories valid integer, invalid

integer, and alpha-special character.

$2*2*2*3 = 24$ (Now, we have only 24 test cases, including negative test cases)

- In order to make text box column, put the most common input on the first place that is a valid integer, on the second place put the second most common input that is an invalid integer, and at the last place put the least common input that is an Alpha Special Character.
- Then start filling the table, the first column is a text box with three values, the next column is a list box that has 2 values, the third column is a checkbox that has 2 values, and the last one is a radio button that also has 2 values.

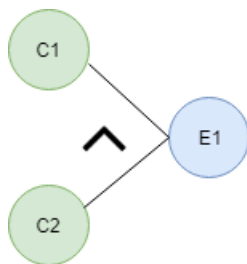
Text Box	List Box	Check Box	Radio Button
Valid Integer	0	Check	ON
Invalid Integer	Other	Uncheck	OFF
Valid Integer	0	Check	ON
Invalid Integer	Other	Uncheck	OFF
AlphaSpecialCharacter	0	Check	ON
AlphaSpecialCharacter	Other	Uncheck	OFF

5. Cause-Effect technique

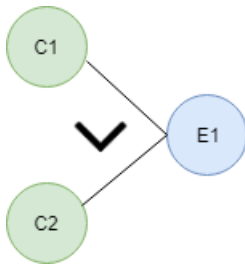
- Cause-Effect Technique underlines the relationship between a given result and all the factors affecting the result.
- Cause-Effect graph technique is based on a collection of requirements and used to determine minimum possible test cases which can cover a maximum test area of the software.
- It is used to write dynamic test cases.
- The dynamic test cases are used when code works dynamically based on user input.
- For example:
- While using email account, on entering valid email, the system accepts it but, when you enter invalid email, it throws an error message.
- In this technique, the input conditions are assigned with causes and the result of these input conditions with effects.
- It reduces the time of test execution and cost.
- To reduce the number of test cases but still covers all necessary test cases with maximum coverage to achieve the desired application quality.
- Cause-Effect graph technique converts the requirements specification into a logical relationship between the input and output conditions by using logical operators like AND, OR and NOT.

Notations used in the Cause-Effect Graph

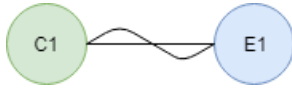
AND - E1 is an effect and C1 and C2 are the causes. If both C1 and C2 are true, then effect E1 will be true.



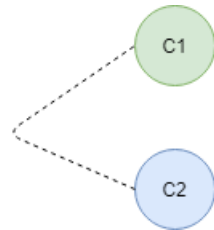
OR - If any cause from C1 and C2 is true, then effect E1 will be true.



NOT - If cause C1 is false, then effect E1 will be true.



Mutually Exclusive - When only one cause is true.



6. Equivalence partitioning technique

- Equivalence partitioning is a technique of software testing in which input data divided into partitions of valid and invalid values, and it is mandatory that all partitions must exhibit the same behavior.
- If a condition of one partition is true, then the condition of another equal partition must also be true, and if a condition of one partition is false, then the condition of another equal partition must also be false.
- The principle of equivalence partitioning is, test cases should be designed to cover each partition at least once.
- Each value of every equal partition must exhibit the same behavior as other.
- The equivalence partitions are derived from requirements and specifications of the software.
- it helps to reduce the time of testing due to a smaller number of test cases from infinite to finite.
- It is applicable at all levels of the testing process.

Example

An OTP number which contains only six digits, less or more than six digits will not be accepted, and the application will redirect the user to the error page.

INVALID	INVALID	VALID	VALID
1 Test case	2 Test case	3 Test case	
DIGITS >=7	DIGITS <=5	DIGITS = 6	DIGITS = 6
93847262	9845	456234	451483

A function of the software application accepts a 10 digit mobile number.

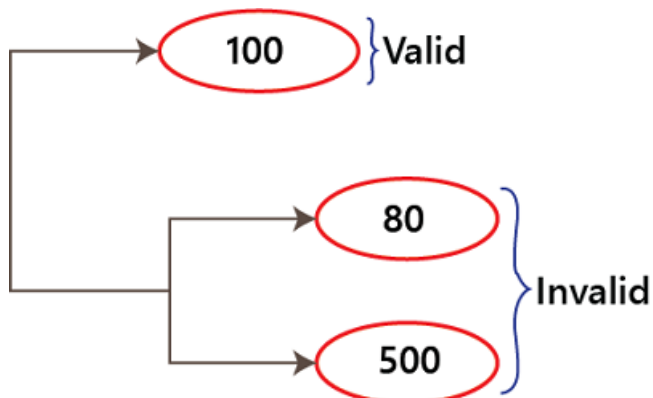
INVALID	INVALID	VALID	VALID
1 Test case	2 Test case	3 Test case	
DIGITS >=11	DIGITS <=9	DIGITS = 10	DIGITS =10
93847262219	984543985	9991456234	9893451483

In both examples, we can see that there is a partition of two equally valid and invalid partitions, on applying valid value such as OTP of six digits in the first example and mobile number of 10 digits in the second example, both valid partitions behave same, i.e. redirected to the next page.

Condition1

If the requirement is a **range of values**, then derive the test case for **one valid** and **two invalid** inputs.

For example, the Amount of test field accepts a Range (100-400) of values:



Example 2

If we are doing online shopping, mobile phone product, and the different **Product ID -1,4,7,9**

ONLINE SHOPING

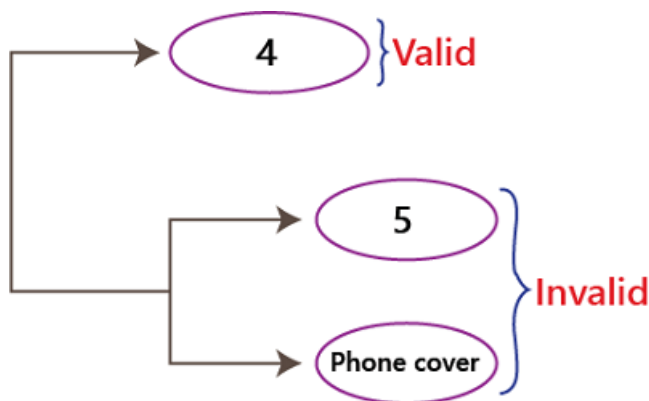
Product ID

SUBMIT

CANCEL

Here, 1 → phone covers 4 → earphones 7 → charger 9 → Screen guard

And if we give the product id as 4, it will be accepted, and it is one valid value, and if we provide the product id as **5 and phone cover**, it will not be accepted as per the requirement, and these are the two invalid values.



7. Error Guessing Technique

- Error guessing is a technique in which there is no specific method for identifying the error. It is based on the experience of the test analyst, where the tester uses the experience to guess the problematic areas of the software.
- It is a type of black box testing technique which does not have any defined structure to find the error.
- In this approach, every test engineer will derive the values or inputs based on their understanding or assumption of the requirements, and we do not follow any kind of rules to perform error guessing technique.
- The error guessing technique is dependent on the ability and product knowledge of the tester because a good test engineer knows where the bugs are most likely to be, which helps to save lots of time.

How does the error guessing technique be implemented?

- The implementation of this technique depends on the experience of the tester or analyst having prior experience with similar applications.
- It requires only well-experienced testers with quick error guessing technique.
- This technique is used to find errors that may not be easily captured by formal black box testing techniques, and that is the reason, it is done after all formal techniques.
- The scope of the error guessing technique entirely depends on the tester and type of experience in the previous testing involvements because it does not follow any method and guidelines.
- Test cases are prepared by the analyst to identify conditions.
- The conditions are prepared by identifying most error probable areas and then test cases are designed for them.
- The main purpose of this technique is to identify common errors at any level of testing.
- For Example:

- Enter blank space into the text fields.
- Null pointer exception.
- Enter invalid parameters.
- Divide by zero.
- Use maximum limit of files to be uploaded.
- Check buttons without entering values.

Purpose of Error guessing

- To deal with all possible errors which cannot be identified informal testing.
- It must contain the all-inclusive sets of test cases without skipping any problematic areas.
- There are some factors that can be used by the examiner while using their experience.

For Example:

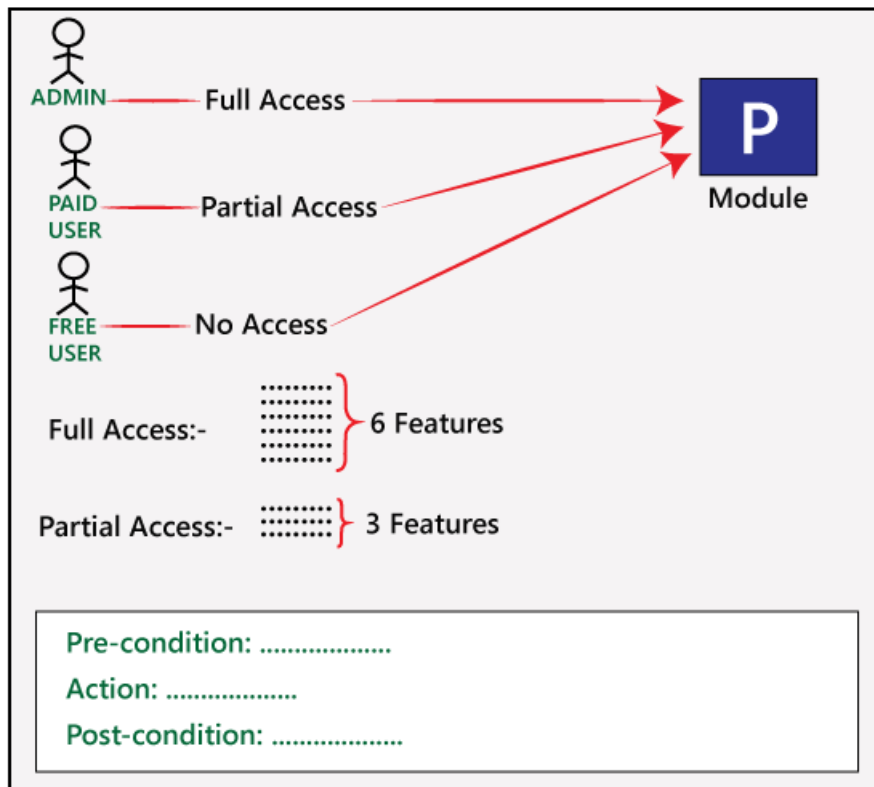
- Historical learning
- Review checklist
- Risk reports of the software
- Previous test results
- Defects occurred in the past

Example1

- A function of the application requires a mobile number which must be of 10 characters. Now, below are the techniques that can be applied to guess error in the mobile number field:
 - What will be the result, if the entered character is other than a number?
 - What will be the result, if entered characters are less than 10 digits?
 - What will be the result, if the mobile field is left blank?
- After implementing these techniques, if the output is similar to the expected result, the function is considered to be bug-free, but if the output is not similar to the expected result, so it is sent to the development team to fix the defects.

8. Use Case Technique

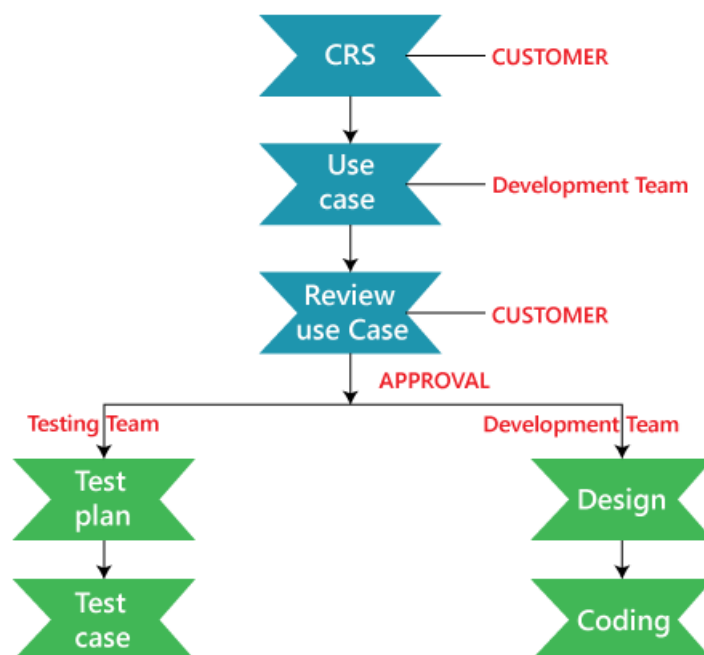
- Use case Technique used to identify the test cases from the beginning to the end of the system as per the usage of the system.
- The test team creates a test scenario that can exercise the entire software based on the functionality of each function from start to end.
- It is a graphic demonstration of business needs, which describe how the end-user will cooperate with the software or the application.
- The use cases provide us all the possible techniques of how the end-user uses the application.



- In the above image, a sample of a use case where we have a requirement related to the customer requirement specification (CRS).
- For module P of the software, we have six different features.
- Admin has access to all the six features, the Paid user has access to the three features and for the free user, there is no access provided to any of the features.

Who writes the use case?

The client provides the customer requirement specification for the application, then the development team will write the **use case** according to the CRS, and the use case is sent to the customer for their review.



If the client approves it, then the approved **use case** is sent to the development team for further design and coding process and these approved use case is also sent to the testing team, so they can start writing the test plan and start writing the test cases for the different features of the software.

Advantages

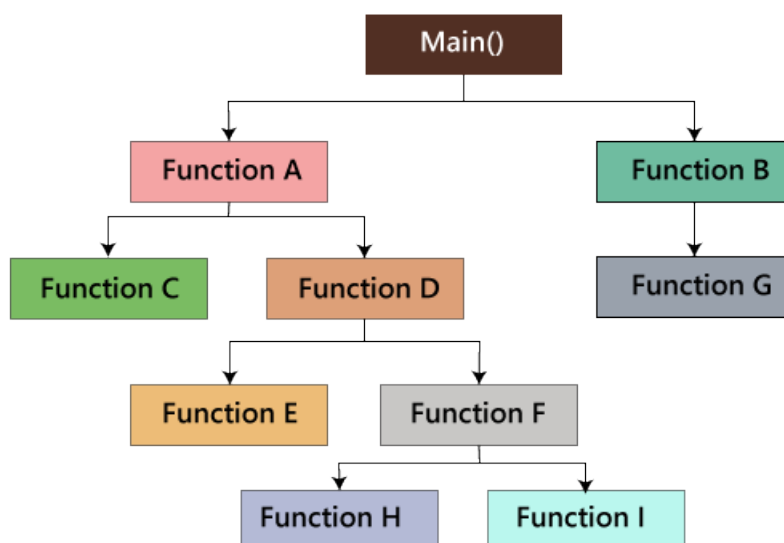
- It is used to take the functional needs of the system.
- Describe the connections between the user and its actions.
- It starts from an elementary view where the system is created first and primarily used for its users.
- It is used to determine the complete analyses, which help us to achieve the complication

WHITE BOX TESTING

- The white box testing also known as glass box is testing, structural testing, clear box testing, open box testing and transparent box testing.
- It tests internal coding and infrastructure of a software focus on checking of predefined inputs against expected and desired outputs.
- It is based on inner workings of an application.
- In this type of testing programming skills are required to design test cases.
- The primary goal of white box testing is to focus on the flow of inputs and outputs through the software and strengthening the security of the software.
- The clear box or white box or transparent box name denote the ability to see through the software's outer shell into its inner workings.
- The developer will test every line of the code of the program.
- The developers perform the White-box testing and then send the application or the software to the testing team.
- They will perform the testing and verify the application along with the requirements and identify the bugs and sends it to the developer.
- The developer fixes the bugs and sends it to the testing team.
- The white box testing contains various tests, which are as follows:
 1. Path testing
 2. Loop testing
 3. Condition testing
 4. Testing based on the memory perspective
 5. Test performance of the program

1. Path Testing

- In the path testing, we will write the flow graphs and test all independent paths.
- Here writing the flow graph implies that flow graphs are representing the flow of the program and also show how every program is added with one another.



- Test all the independent paths, a path from main() to function G, first set the parameters and test if the program is correct in that particular path, and in the same way test all other paths and fix the bugs.

2. Loop Testing

- In the loop testing, we will test the loops such as while, for, and do-while, etc. and also check for ending condition if working correctly and if the size of the conditions is enough.

For example: we have one program where the developers have given about 50,000 loops.

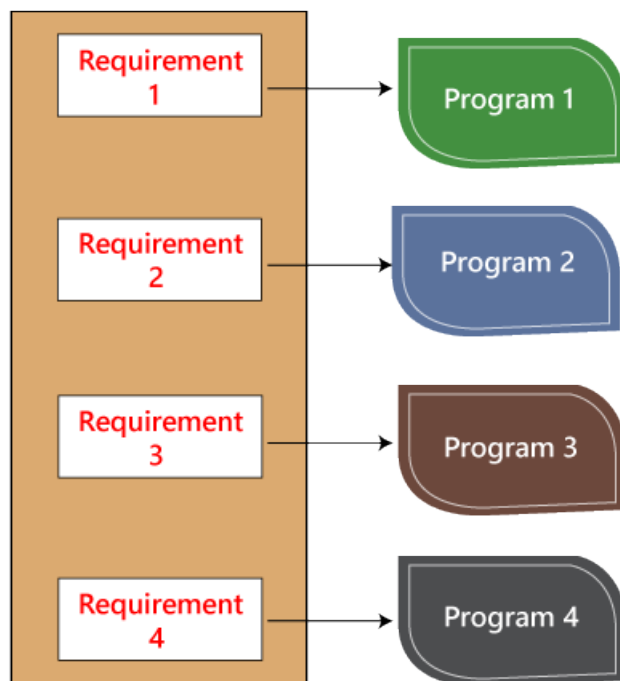
```
{
while(50,000)
.....
.....
}
```

- We cannot test this program manually for all the 50,000 loops cycle.
- So we write a small program that helps for all 50,000 cycles that test P is written in the similar language as the source code program, and this is known as a Unit test. And it is written by the developers only.

Test P

```
{
.....
..... }
```

- We have various requirements such as 1, 2, 3, and 4. , then, the developer writes the programs such as program 1,2,3,4 for the parallel conditions. Here the application contains the 100s line of codes.

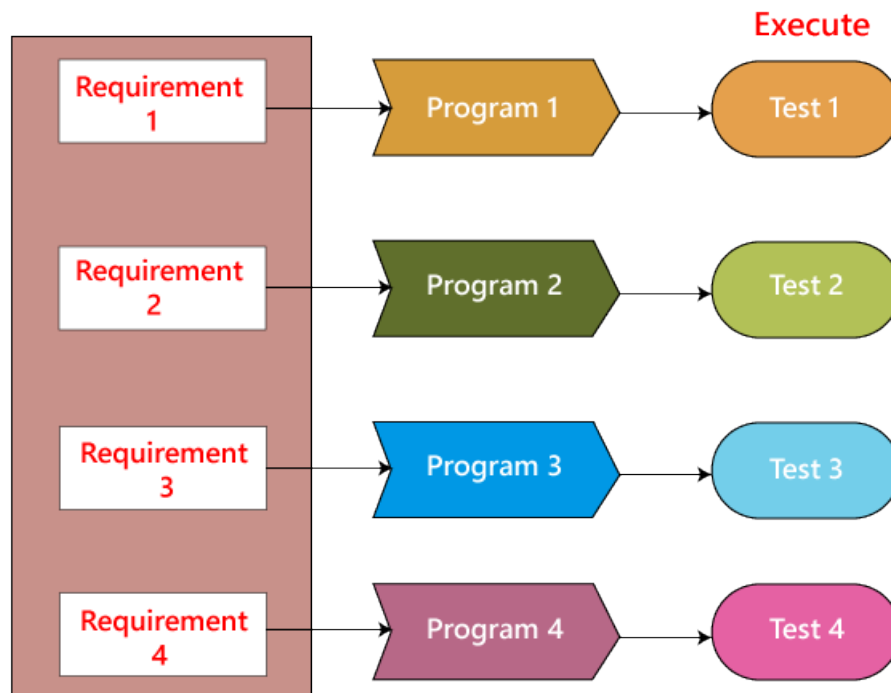


- The developer will do the white box testing, and they will test all the programs line by line of code to find the bug.
- If they found any bug in any of the programs, they will correct it.
- Again have to test the system then this process contains lots of time and effort and slows down the product release time.
- The another case, where the clients want to modify the requirements, then the developer will do the required changes and test all four program again, which take lots of time and efforts.

These issues can be resolved in the following ways:

- In this, we will write test for a similar program where the developer writes these test code in the related language as the source code.

- Then they execute these test code, which is also known as unit test programs.
- These test programs linked to the main program and implemented as programs.



- Therefore, if there is any requirement of modification or bug in the code, then the developer makes the adjustment both in the main program and the test program and then executes the test program.

3. Condition Testing

- In this, we will test all logical conditions for both **true** and **false** values; that is, we will verify for both **if** and **else** condition.

For example:

```

if(condition) - true
{
.....
.....
.....
}
else - false
{
.....
.....
.....
}

```

- The above program will work fine for both the conditions, which means that if the condition is accurate.

4. Testing based on the memory (size) perspective

The size of the code is increasing for the following reasons:

- **The reuse of code is not there:** let us take one example, where we have four programs of the same application, and the first ten lines of the program are similar. We can write these ten lines as a discrete function, and it should be accessible by the above four programs as well. And also, if any bug is there, we can modify the line of code in the function rather than the entire code.
- **The developers use the logic** that might be modified. If one programmer writes code and the file size is up to 250kb, then another programmer could write a similar code using the different logic, and the file size is up to 100kb.
- **The developer declares so many functions and variables** that might never be used in any portion of the code. Therefore, the size of the program will increase.

For example,

```
int a=15;
int b=20;
String S= "Welcome";
....
.....
int p=b;
Create user()
{
.....
.....
200's line of code
}
```

- In the above code, we can see that the integer a has never been called anywhere in the program, and also the function Create user has never been called anywhere in the code. Therefore, it leads us to memory consumption.

5. Test the performance (Speed, response time) of the program

The application could be slow for the following reasons:

- When logic is used.
- For the conditional cases, we will use **or & and** adequately.
- Switch case, which means we cannot use **nested if**, instead of using a switch case.

The developer is performing white box testing, they understand that the code is running slow, or the performance of the program is also getting deliberate. And the developer cannot go manually over the program and verify which line of the code is slowing the program.

- To recover with this condition, we have a tool called **Rational Quantify**, which resolves these kinds of issues automatically.
- Once the entire code is ready, the rational quantify tool will go through the code and execute it.
-

TECHNIQUES USED IN WHITE BOX TESTING

1. Data Flow Testing
2. Control Flow Testing
3. Branch Testing
4. Statement Testing
5. Decision Testing

1. Data Flow Testing

- **Data flow testing is used to analyze the flow of data in the program.**
- **It is the process of collecting information about how the variables flow the data in the program.**
- Data flow testing is a group of testing strategies to examine the control flow of programs in order to explore the sequence of variables according to the sequence of events.
- It mainly focuses on the points at which values assigned to the variables and the point at which these values are used by concentrating on both points, data flow can be tested.
- Data flow testing uses the control flow graph to detect illogical things that can interrupt the flow of data.
- Anomalies in the flow of data are detected at the time of associations between values and variables due to:
 - If the variables are used without initialization.
 - If the initialized variables are not used at least once.

Example

1. read x;	
2. If(x>0)	(1, (2, t), x), (1, (2, f), x)
3. a= x+1	(1, 3, x)
4. if (x<=0) {	(1, (4, t), x), (1, (4, f), x)
5. if (x<1)	(1, (5, t), x), (1, (5, f), x)
6. x=x+1; (go to 5)	(1, 6, x)
else	
7. a=x+1	(1, 7, x)
8. print a;	(6,(5, f)x), (6,(5,t)x)
	(6, 6, x)
	(3, 8, a), (7, 8, a).

- In this code, we have a total 8 statements, and we will choose a path which covers all the 8 statements.
- We cannot cover all the statements in a single path because if statement 2 is true then statements 4, 5, 6, 7 not covered, and if statement 4 is true then statement 2 and 3 are not covered.
- So, we are taking two paths to cover all the statements.

1. x= 1

Path - 1, 2, 3, 8

- When we set value of x as 1 first it come on step 1 to read and assign the value of x (we took 1 in path) then come on statement 2 (x>0 (we took 2 in path)) which is true and it comes on statement 3 (a= x+1 (we took 3 in path)) at last it comes on statement 8 to print the value of x (output is 2).

2. Set x= -1

Path = 1, 2, 4, 5, 6, 5, 6, 5, 7, 8

- When we set the value of x as -1 then first, it comes on step 1 to read and assign the value of x , then come on step number 2 which is false because x is not greater than 0.
- Due to false condition, it will not come on statement 3 and directly jump on statement 4 and 4 is , then come on statement 5 , which is also true so it will come on statement 6 (x=x+1 and here x is incremented by 1.

x=-1+1

x=0

- There is value of x become 0. Now it goes to statement 5 with value 0 and 0 is less than 1 so, it is true. Come on statement 6.

x=x+1

x= 0+1

x=1

- There x has become 1 and again goes to statement and now 1 is not less than 1 so, condition is false and it will come to else part and assign the value to a .
- At last, it come on statement 8 and print the value (Output is 2).

Associations

In associations all the definitions with all of its uses.

How to make associations in data flow testing?

1 read x;	
2 If(x>0)	(1, (2, t), x), (1, (2, f), x)
3 a= x+1	(1, 3, x)
4 if (x<=0) {	(1, (4, t), x), (1, (4, f), x)
5 if (x<1)	(1, (5, t), x), (1, (5, f), x)
6 x=x+1; (go to 5)	(1, 6, x)
else	
7 a=x+1	(1, 7, x)
8 print a;	(6,(5, f)x), (6,(5,t)x)
	(6, 6, x)
	(3, 8, a), (7, 8, a).

- (1, (2, t), x), (1, (2, f), x)- This association is made with statement 1 (read x;) and statement 2 (If(x>0)) where x is defined at line number 1, and it is used at line number 2 so, x is the variable.
- Statement 2 is logical, and it can be true or false that's why the association is defined in two ways; one is (1, (2, t), x) for true and another is (1, (2, f), x) for false.
- (1, 3, x)- This association is made with statement 1 (read x;) and statement 3 (a= x+1) where x is defined in statement 1 and used in statement 3. It is a computation use.

2. Control Flow Testing

- Control flow testing is a testing technique that comes under white box testing.
- To determine the execution order of statements or instructions of the program through a control structure.
- The control structure of a program is used to develop a test case for the program.
- In this technique, a particular part of a large program is selected by the tester to set the testing path.
- It is mostly used in unit testing. Test cases represented by the control graph of the program.
- Control Flow Graph is formed from the node, edge, decision node, junction node to specify all possible execution path.
- Notations used for Control Flow Graph
 1. Node
 2. Edge
 3. Decision Node
 4. Junction node

1. Node

- Nodes in the control flow graph are used to create a path of procedures.
- It represents the sequence of procedures.
- The tester can determine the sequence of occurrence of procedures.

2. Edge

- Edge in control flow graph is used to link the direction of nodes.

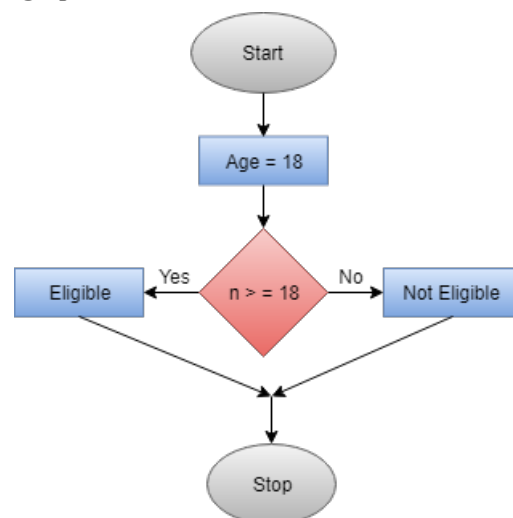
3. Decision node

- Decision node in the control flow graph is used to decide next node of procedure as per the value.

4. Junction node

- Junction node in control flow graph is the point where at least three links meet.

Diagram - control flow graph



- Test cases are designed through the flow graph of the programs to determine the execution path is correct or not.
- All nodes, junction, edges, and decision are the essential parts to design test cases.

3. Branch Coverage Testing

- Branch coverage technique is used to cover all branches of the control flow graph.
- It covers all the possible outcomes (true and false) of each condition of decision point at least once.
- It ensures that every branch of each decision point must be executed.

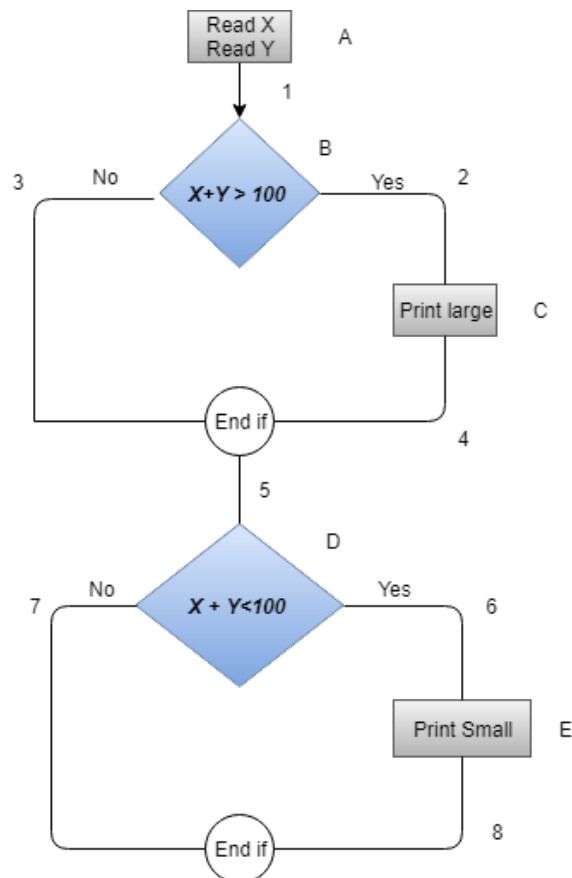
How to calculate Branch coverage?

- There are several methods to calculate Branch coverage, but path finding is the most common method.
- The number of paths of executed branches is used to calculate Branch coverage.
- Branch coverage technique can be used as the alternative of decision coverage.

Example

1. Read X
2. Read Y
3. IF $X+Y > 100$ THEN
4. Print "Large"
5. ENDIF
6. If $X + Y < 100$ THEN
7. Print "Small"
8. ENDIF

Control flow graph of code structure



- In the first case traversing through "Yes" decision, the path is **A1-B2-C4-D6-E8**, and the number of covered edges is 1, 2, 4, 5, 6 and 8 but edges 3 and 7 are not covered in this path.
- To cover these edges, to traverse through "No" decision.

- In the case of "No" decision the path is **A1-B3-5-D7**, and the number of covered edges is 3 and 7.
Path 1 - A1-B2-C4-D6-E8
Path 2 - A1-B3-5-D7
Branch Coverage (BC) = Number of paths
=2

Software Testing Life Cycle (STLC)

- The Software Testing Life Cycle (STLC) is a systematic approach to testing a software application to ensure that it meets the requirements and is free of defects.
- It is a process that follows a series of steps or phases, and each phase has specific objectives and deliverables.
- The STLC is used to ensure that the software is of high quality, reliable, and meets the needs of the end-users.
- The main goal of the STLC is to identify and document any defects or issues in the software application as early as possible in the development process.
- This allows for issues to be addressed and resolved before the software is released to the public.
- Stages of the STLC include Test Planning, Test Analysis, Test Design, Test Environment Setup, Test Execution, Test Closure, and Defect Retesting.
- Each of these stages includes specific activities and deliverables that help to ensure that the software is thoroughly tested.

Phases of STLC

1. Requirement Analysis: Requirement Analysis is the first step of the Software Testing Life Cycle (STLC). In this phase quality assurance team understands the requirements like what is to be tested. If anything is missing or not understandable then the quality assurance team meets with the stakeholders to better understand the detailed knowledge of requirements.

The activities that take place during the Requirement Analysis stage include:

- Reviewing the software requirements document (SRD) and other related documents
- Interviewing stakeholders to gather additional information
- Identifying any ambiguities or inconsistencies in the requirements
- Identifying any missing or incomplete requirements
- Identifying any potential risks or issues that may impact the testing process

Creating a requirement traceability matrix (RTM) to map requirements to test cases.

The testing team should have a clear understanding of the software requirements and should have identified any potential issues that may impact the testing process.

This will help to ensure that the testing process is focused on the most important areas of the software and that the testing team is able to deliver high-quality results.

2. Test Planning: Test Planning is the most efficient phase of the software testing life cycle where all testing plans are defined. In this phase manager of the testing, team calculates the estimated effort and cost for the testing work. This phase gets started once the requirement-gathering phase is completed.

The activities that take place during the Test Planning stage include:

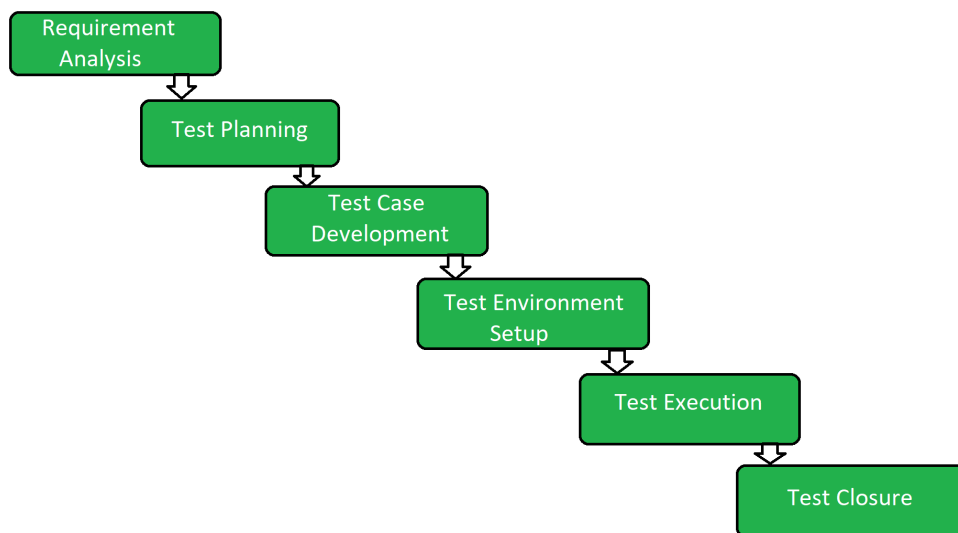
- Identifying the testing objectives and scope
- Developing a test strategy: selecting the testing methods and techniques that will be used
- Identifying the testing environment and resources needed
- Identifying the test cases that will be executed and the test data that will be used
- Estimating the time and cost required for testing
- Identifying the test deliverables and milestones
- Assigning roles and responsibilities to the testing team
- Reviewing and approving the test plan

3. Test Case Development: The test case development phase gets started once the test planning phase is completed. In this phase testing team notes down the detailed test cases. The testing team also prepares the required test data for the testing. When the test cases are prepared then they are reviewed by the quality assurance team.

The activities that take place during the Test Case Development stage include:

- Identifying the test cases that will be developed
- Writing test cases that are clear, concise, and easy to understand
- Creating test data and test scenarios that will be used in the test cases
- Identifying the expected results for each test case
- Reviewing and validating the test cases
- Updating the requirement traceability matrix (RTM) to map requirements to test cases

Software Testing Lifecycle



4. Test Environment Setup:

Test environment setup is a vital part of the STLC.

Basically, the test environment decides the conditions on which software is tested.

In this process, the testing team is not involved either the developer or the customer creates the testing environment.

5. Test Execution:

After the test case development and test environment setup test execution phase gets started.

In this phase testing team starts executing test cases based on prepared test cases in the earlier step.

The activities that take place during the test execution stage of the Software Testing Life Cycle (STLC) include:

- **Test execution:** The test cases and scripts created in the test design stage are run against the software application to identify any defects or issues.
- **Defect logging:** Any defects or issues that are found during test execution are logged in a defect tracking system, along with details such as the severity, priority, and description of the issue.
- **Test data preparation:** Test data is prepared and loaded into the system for test execution
- **Test environment setup:** The necessary hardware, software, and network configurations are set up for test execution
- **Test execution:** The test cases and scripts are run, and the results are collected and analyzed.

- **Test result analysis:** The results of the test execution are analyzed to determine the software's performance and identify any defects or issues.
- **Defect retesting:** Any defects that are identified during test execution are retested to ensure that they have been fixed correctly.
- **Test Reporting:** Test results are documented and reported to the relevant stakeholders.

It is important to note that test execution is an iterative process and may need to be repeated multiple times until all identified defects are fixed and the software is deemed fit for release.

6. Test Closure: Test closure is the final stage of the Software Testing Life Cycle (STLC) where all testing-related activities are completed and documented. The main objective of the test closure stage is to ensure that all testing-related activities have been completed and that the software is ready for release.

At the end of the test closure stage, the testing team should have a clear understanding of the software's quality and reliability, and any defects or issues that were identified during testing should have been resolved.

The test closure stage also includes documenting the testing process and any lessons learned so that they can be used to improve future testing processes.

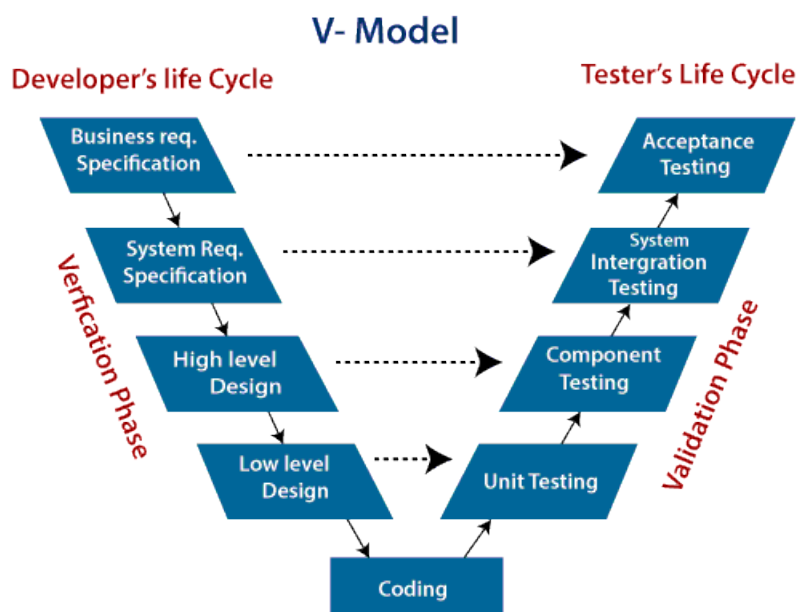
Test closure is the final stage of the Software Testing Life Cycle (STLC) where all testing-related activities are completed and documented.

The main activities that take place during the test closure stage include:

- **Test summary report:** A report is created that summarizes the overall testing process, including the number of test cases executed, the number of defects found, and the overall pass/fail rate.
- **Defect tracking:** All defects that were identified during testing are tracked and managed until they are resolved.
- **Test environment clean-up:** The test environment is cleaned up, and all test data and test artifacts are archived.
- **Test closure report:** A report is created that documents all the testing-related activities that took place, including the testing objectives, scope, schedule, and resources used.
- **Knowledge transfer:** Knowledge about the software and testing process is shared with the rest of the team and any stakeholders who may need to maintain or support the software in the future.
- **Feedback and improvements:** Feedback from the testing process is collected and used to improve future testing processes.

V-Model

V-Model also referred to as the Verification and Validation Model. In this, each phase of SDLC must complete before the next phase starts. It follows a sequential design process same as the waterfall model. Testing of the device is planned in parallel with a corresponding stage of development.



Verification:

It involves a static analysis method (review) done without executing code. It is the process of evaluation of the product development process to find whether specified requirements meet.

Validation:

It involves dynamic analysis method (functional, non-functional), testing is done by executing code. Validation is the process to classify the software after the completion of the development process to determine whether the software meets the customer expectations and requirements.

There are the various phases of Verification Phase of V-model:

1. **Business requirement analysis:** This is the first step where product requirements understood from the customer's side. This phase contains detailed communication to understand customer's expectations and exact requirements.
2. **System Design:** In this stage system engineers analyze and interpret the business of the proposed system by studying the user requirements document.
3. **Architecture Design:** The baseline in selecting the architecture is that it should understand all which typically consists of the list of modules, brief functionality of each module, their interface relationships, dependencies, database tables, architecture diagrams, technology detail, etc.
4. **Module Design:** In the module design phase, the system breaks down into small modules. The detailed design of the modules is specified, which is known as Low-Level Design
5. **Coding Phase:** After designing, the coding phase is started. Based on the requirements, a suitable programming language is decided.

There are the various phases of Validation Phase of V-model:

1. **Unit Testing:** In the V-Model, Unit Test Plans (UTPs) are developed during the module design phase. These UTPs are executed to eliminate errors at code level or unit level. Unit testing verifies that the smallest entity can function correctly when isolated from the rest of the codes/ units.
2. **Integration Testing:** Integration Test Plans are developed during the Architectural Design Phase. These tests verify that groups created and tested independently can coexist and communicate among themselves.
3. **System Testing:** System Tests Plans are developed during System Design Phase. System Test ensures that expectations from an application developer are met.
4. **Acceptance Testing:** Acceptance testing is related to the business requirement analysis part. It includes testing the software product in user level.

Advantage:

1. Easy to Understand.
2. Testing Methods like planning, test designing happens well before coding.
3. Saves a lot of time.
4. Avoids the downward flow of the defects.
5. Works well for small plans where requirements are easily understood.

Disadvantage:

1. Very rigid and least flexible.
2. Not a good for a complex project.
3. Software is developed during the implementation stage, so no early prototypes of the software are produced.
4. If any changes happen in the midway, then the test documents along with the required documents, has to be updated.

Program Correctness

The best practices to ensure software correctness are listed below –

- The problem statement or the use case for the software should be clear and unambiguous.

- Before actually implementing the code, there should be a complete architecture, and design for the software.
- Testing should be done in parallel to the development of the software.

Software Reliability & Safety

Reliability focuses on a system's ability to perform its intended function consistently over time, while safety focuses on preventing potential harm or damage caused by the software's operation.

Software Failures

- To analyze software reliability and safety it is important to understand the meaning of a failure due to software.
- Software does not exhibit the random or wear out related failure behavior.
- Software will always function in the same way as long as the same input and computer states are present.
- Software can cause system failures either because of design errors or implementation errors.
- Design errors are often caused by wrong assumptions about system operation.

Example:

Input A is always followed by input B. Typical implementation errors are caused by confusing symbols, such as 'g' instead of 'G'. Software faults can only cause failures if the fault is encountered during usage. Therefore faults existing in frequently used code will cause failures more often than faults residing in rarely used code but the later can be equally serious. In mission or safety critical applications it is particularly important to review and test rarely used code.

Software Reliability

- Software reliability involves much more than analyzing test results, estimating remaining faults, and modeling future failure probabilities.
- Although in most organizations software test is no longer an afterthought, management is almost always surprised by the cost and schedule requirements of the test program, and it is often downgraded in favor of design activities.
- Often adding a new feature will seem more beneficial than performing a complete test on existing features.

A good software reliability program, introduced early in the development cycle, will mitigate these problems by:

- Preparing program management in advance for the testing effort and allowing them to plan both schedule and budget to cover the required testing.
- Continuous review of requirements throughout the life cycle, particularly for handling of exception conditions. If requirements are incomplete there will be no testing of the exception conditions.
- Offering management a quantitative assessment of the dependence of reliability metrics (software/system availability; software/system outages per day, etc.) on the effort (time and cost) allotted to testing.
- Providing the most efficient test plan targeted to bringing the product to market in the shortest time subject to the reliability requirements imposed by the customer or market expectations.
- Performing continuous quantitative assessment of software/system reliability and the effort/cost required to improve them by a specified amount.

Software Safety

As systems and products become more and more dependent on software components it is no longer realistic to develop a system safety program that does not include the software elements.

Safety-critical software

Safety-critical software is very different from both non-critical software and safety-critical hardware. The difference lies in the massive testing program that such software undergoes.

What are "software failure modes"?

- Software, especially in critical systems, tends to fail where least expected.

- We are usually extremely good at setting up test plans for the main line code of the program, and these sections usually do run flawlessly.
- Software does not "break" but it must be able to deal with "broken" input and conditions, which often cause the "software failures".
- The task of dealing with abnormal/anomalous conditions and inputs is handled by the exception code dispersed throughout the program.
- Setting up a test plan and exhaustive test cases for the exception code is by definition difficult and somewhat subjective.

Software Testing – Bug vs Defect vs Error vs Fault vs Failure

- Software Testing defines a set of procedures and methods that check whether the actual software product matches with expected requirements, thereby ensuring that the product is Defect free.
- There are a set of procedures that needs to be in mind while testing the software manually or by using automated procedures.
- The main purpose of software testing is to identify errors, deficiencies, or missing requirements with respect to actual requirements.
- Software Testing is Important because if there are any bugs or errors in the software, they can be identified early and can be solved before the delivery of the software product.

What is a Bug?

A bug refers to defects which means that the software product or the application is not working as per the adhered requirements set. When we have any type of logical error, it causes our code to break, which results in a bug. It is now that the Automation/Manual Test Engineers describe this situation as a bug.

What is a Defect?

A defect refers to a situation when the application is not working as per the requirement and the actual and expected result of the application or software are not in sync with each other.

What is an Error?

Error is a situation that happens when the Development team or the developer fails to understand a requirement definition and hence that misunderstanding gets translated into buggy code. This situation is referred to as an Error and is mainly a term coined by the developers.

What is a Fault?

Fault occurs in software which means that the logic was not incorporated to handle the errors in the application. This is an undesirable situation, but it mainly happens due to invalid documented steps or a lack of data definitions.

What is a Failure?

Failure is the accumulation of several defects that ultimately lead to Software failure and results in the loss of information in critical modules thereby making the system unresponsive. Generally, such situations happen very rarely because before releasing a product all possible scenarios and test cases for the code are simulated. Failure is detected by end-users once they face a particular issue in the software.

- Failure can happen due to human errors or can also be caused intentionally in the system by an individual.
- It is a term that comes after the production stage of the software.
- It can be identified in the application when the defective part is executed.

A simple diagram depicting Bug vs Defect vs Fault vs Failure:



Bug vs Defect vs Error vs Fault vs Failure

Basis	Bug	Defect	Fault	Error	Failure
Definition	A bug refers to defects which means that the software product or the application is not working as per the adhered requirements set	A Defect is a deviation between the actual and expected output	A Fault is a state that causes the software to fail and therefore it does not achieve its necessary function.	An Error is a mistake made in the code due to which compilation or execution fails,	Failure is the accumulation of several defects that ultimately lead to Software failure and results in the loss of information in critical modules thereby making the system unresponsive.
Raised by	Test Engineers	The defect is identified by The Testers And is resolved by developers in the development phase of SDLC.	Human mistakes lead to fault.	Developers and automation test engineers	The failure is found by the test engineer during the development cycle of SDLC
Different types	• Logical bugs • Algorithmic bugs • Resource bugs	Defects are classified as follows: Based on Priority: • High • Medium • Low Based on Severity: • Critical • Major • Minor • Trivial	• Business Logic Faults • Functional and Logical Faults • Graphical User Interface (GUI) Faults • Performance Faults • Security Faults • Hardware Faults	• Syntactic Error • UI screen error • Error handling error • Flow control error • Calculation error • Hardware error	NA
	• Missing Logic • Erroneous Logic • Redund	• Receiving & providing incorrect input • Coding/ Logical Error	• Wrong design of the data definition processes. • An irregularity in Logic or gaps in the	• Error in code. • Inability to compile/execute a program • Ambiguity in code logic • Misunderstanding of	• Environment variables • System Errors

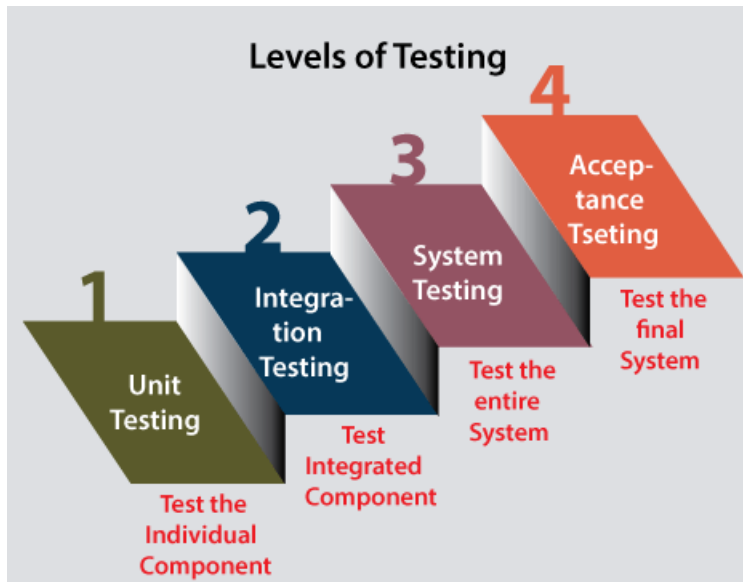
Reasons behind	ant codes	leading to the breakdown of software	software leads to the non-functioning of the software.	requirements • Faulty design and architecture • Logical error	• Human Error
Way to prevent the reasons	- Implementing Test-driven development. - Adjusting enhanced development practices and evaluation of cleanliness of the code.	- Implementing Out-of-the-box programming methods. - Proper usage of primary and correct software coding practices.	- Peer review of the Test documents and requirements. - Verifying the correctness of software design and coding.	- Conduct peer reviews and code-reviews - Need for validation of bug fixes and enhancing the overall quality of the software.	- Confirmation of Re-testing the process end to end, - Carefully review the requirements as well as the specifications. - Categorizing and evaluating the errors and issues.

Stages of Software Testing

The levels of software testing involve the different methodologies, which can be used while we are performing the software testing.

In [software testing](#), we have four different levels of testing, which are as discussed below:

1. **Unit Testing**
2. **Integration Testing**
3. **System Testing**
4. **Acceptance Testing**



Unit Testing

Unit testing is the first level of software testing, which is used to test if software modules are satisfying the given requirement or not.

The first level of testing involves **analyzing each unit or an individual component** of the software application.

Unit testing is also the first level of **functional testing**. The primary purpose of executing unit testing is to validate unit components with their performance.

A unit component is an individual function or regulation of the application, or we can say that it is the smallest testable part of the software. The reason of performing the unit testing is to test the correctness of inaccessible code.

Unit testing will help the test engineer and developers in order to understand the base of code that makes them able to change defect causing code quickly. The developers implement the unit.

Some crucial reasons are listed below:

- Unit testing helps tester and developers to understand the base of code that makes them able to change defect causing code quickly.
- Unit testing helps in the documentation.
- Unit testing fixes defects very early in the development phase that's why there is a possibility to occur a smaller number of defects in upcoming testing levels.
- It helps with code reusability by migrating code and test cases.

Example of Unit testing

Amount transfer

Below are the application access details, which is given by the customer

- URL → login Page
- Username/password/OK → home page
- To reach Amount transfer module follow the below

Loans → sales → Amount transfer

While performing unit testing, we should follow some rules, which are as follows:

- To start unit testing, at least we should have one module.
- Test for positive values
- Test for negative values
- No over testing
- No assumption required

When we feel that the **maximum test coverage** is achieved, we will stop the testing.

Now, we will start performing the unit testing on the different components such as

- **From account number(FAN)**

- **To account number(TAN)**

- **Amount**

- **Transfer**

- **Cancel**

Values

Description

1234	accept
4311	Error message→ account valid or not
blank	Error message→ enter some values
5 digit/ 3 digit	Error message→ accept only 4 digit
Alphanumeric	Error message → accept only digit
Blocked account no	Error message
Copy and paste the value	Error message→ type the value
Same as FAN and TAN	Error message

For the TAN component

- Provide the values just like we did in **From account number (FAN)** components

For Amount component

- Provide the values just like we did in FAN and TAN components.

For Transfer component

- Enter valid FAN value
- Enter valid TAN value
- Enter the correct value of Amount
- Click on the Transfer button→ amount transfer successfully(confirmation message)

For Cancel Component

- Enter the values of FAN, TAN, and amount.
- Click on the Cancel button → all data should be cleared.

Unit Testing Tools

We have various types of unit testing tools available in the market, which are as follows:

- NUnit
- JUnit
- PHPunit
- Parasoft Jtest
- EMMA

Unit Testing Techniques:

Unit testing uses all white box testing techniques as it uses the code of software application:

- Data flow Testing
- Control Flow Testing
- Branch Coverage Testing

- Statement Coverage Testing
- Decision Coverage Testing

How to achieve the best result via Unit testing?

Unit testing can give best results without getting confused and increase complexity by following the steps listed below:

- Test cases must be independent because if there is any change or enhancement in requirement, the test cases will not be affected.
- Naming conventions for unit test cases must be clear and consistent.
- During unit testing, the identified bugs must be fixed before jump on next phase of the SDLC.
- Only one code should be tested at one time.
- Adopt test cases with the writing of the code, if not doing so, the number of execution paths will be increased.
- If there are changes in the code of any module, ensure the corresponding unit test is available or not for that module.

Integration Testing

The second level of software testing is the integration testing. The integration testing process comes after unit testing.

It is mainly used to test the data flow from one module or component to other modules.

In integration testing, the **test engineer** tests the units or separate components or modules of the software in a group.

The primary purpose of executing the integration testing is to identify the defects at the interaction between integrated components or units.

When each component or module works separately, we need to check the data flow between the dependent modules, and this process is known as **integration testing**.

Integration testing aims to evaluate the accuracy of communication among all the modules.

Example of integration testing

Let us assume that we have a **Gmail** application where we perform the integration testing.

First, we will do **functional testing** on the **login page**, which includes the various components such as **username, password, submit, and cancel** button. Then only we can perform integration testing.

The different integration scenarios are as follows:

The image shows a screenshot of the Gmail 'Compose Mail' interface. On the left is a sidebar with a list of folders: Inbox, Compose mail, Sent Items, Trash, Spam, Contact, Folders, and Logout. The main area is titled 'Compose Mail' and contains the following elements: 'To' and 'From' labels with text input fields, a 'Subject' label with a text input field, a large 'TEXT FIELD' for the body of the email, two checkboxes labeled 'Save To Draft' and 'Add To Contact', and two buttons at the bottom: 'SEND' and 'CANCEL'.

Scenario 1:

- First, we login as **P** users and click on the **Compose** mail and performing the functional testing for the specific components.
- Now we click on the **Send** and also check for **Save Drafts**.
- After that, we send a **mail** to **Q** and verify in the **Send Items** folder of P to check if the send mail is there.
- Now, we will **log out** as P and login as Q and move to the **Inbox** and verify that if the mail has reached.

Scenario 2:

We also perform the integration testing on **Spam** folders. If the particular contact has been marked as spam, then any mail sent by that user should go to the spam folder and not in the inbox.

As we can see in the below image, we will perform the **functional testing** for all the **text fields and every feature**. Then we will perform **integration testing** for the related functions. We first test the **add user, list of users, delete user, edit user**, and then **search user**.

Integration Testing Techniques

Any testing technique (Blackbox, Whitebox, and Greybox) can be used for Integration Testing; some are listed below:

Black Box Testing

- State Transition technique
- Decision Table Technique
- Boundary Value Analysis
- All-pairs Testing
- Cause and Effect Graph
- Equivalence Partitioning
- Error Guessing

White Box Testing

- Data flow testing
- Control Flow Testing
- Branch Coverage Testing
- Decision Coverage Testing

System Testing

The third level of software testing is **system testing**, which is used to test the software's functional and non-functional requirements.

It is end-to-end testing where the testing environment is parallel to the production environment. In the third level of software testing, we will test the application as a whole system.

To check the end-to-end flow of an application or the software as a user is known as **System testing**.

In system testing, we will go through all the necessary modules of an application and test if the end features or the end business works fine, and test the product as a complete system.

System testing is a sequence of different types of tests to implement and examine the entire working of an integrated software computer system against requirements.

Example of System testing

Suppose we open an application, let say **www.rediff.com**, and there we can see that an advertisement is displayed on the top of the homepage, and it remains there for a few seconds before it disappears. These types of Ads are done by the Advertisement Management System (AMS). Now, we will perform system testing for this type of field.

The below application works in the following manner:

- Let's say that Amazon wants to display a promotion ad on January 26 at precisely 10:00 AM on the Rediff's home page for the country India.
- Then, the sales manager logs into the website and creates a request for an advertisement dated for the above day.
- He/she attaches a file that likely an image files or the video file of the AD and applies.
- The next day, the AMS manager of Rediffmail login into the application and verifies the awaiting Ad request.
- The AMS manager will check those Amazons ad requests are pending, and then he/she will check if the space is available for the particular date and time.
- If space is there, then he/she evaluate the cost of putting up the Ad at 15\$ per second, and the overall Ad cost for 10 seconds is approximate 150\$.
- The AMS manager clicks on the payment request and sends the estimated value along with the request for payment to the Amazon manager.
- Then the amazon manager login into the Ad status and confirms the payment request, and he/she makes the payment as per all the details and clicks on the **Submit** and **Pay**
- As soon as Rediff's AMs manager gets the amount, he/she will set up the Advertisement for the specific date and time on the Rediffmail's home page.
- **Scenario1:** The first test is the general scenario, as we discussed above. The test engineer will do the system testing for the underlying situation where the Amazon manager creates a request for the Ad and that Ad is used at a particular date and time.
- **Scenario2:** Suppose the Amazon manager feels that the AD space is too expensive and cancels the request. At the same time, the Flipkart requests the Ad space on January 26 at 10:00 AM. Then the request of Amazon has been canceled. Therefore, Flipkart's promotion ad must be arranged on January 26 at 10 AM.
- After all, the request and payment have been made. Now, if Amazon changes their mind and they feel that they are ready to make payment for January 26 at 10 AM, which should be given because Flipkart has already used that space. Hence, another calendar must open up for Amazon to make their booking.
- **Scenario3:** in this, first, we login as AMS manger, then click on Set Price page and set the price for AD space on logout page to 10\$ per second.
- Then login as Amazon manager and select the date and time to put up and Ad on the logout page. And the payment should be 100\$ for 10 seconds of an Ad on Rediffmail logout page.

Name	Status	Total Amount
Amazon	Available	150\$
Flipkart	Available	100\$
.....		
.....		

Pay
Cancel

Credit Card No.

SUBMIT
CANCEL

Request for Advertisement
AD Status

REQUEST FOR ADVERTISEMENT

Product

Country

China
Russia
U.S.A

Page

Sent Items
Compose Mail
Logout

Duration

Date

Attach Advertisement
[Text,Audio,Image,Video]

SUBMIT
CANCEL

PENDING ADVERTISEMENT

NAME	STATUS	SELECT
.....		☐
.....		☐
.....		☐

Pending Ads
Set Rates

Amazon	Pending
Flipkart	Pending

CHECK AVAILABILITY
PAYMENT REQUEST

APPROXIMATE
SETUP

SET PRICE

Country

Page

Duration

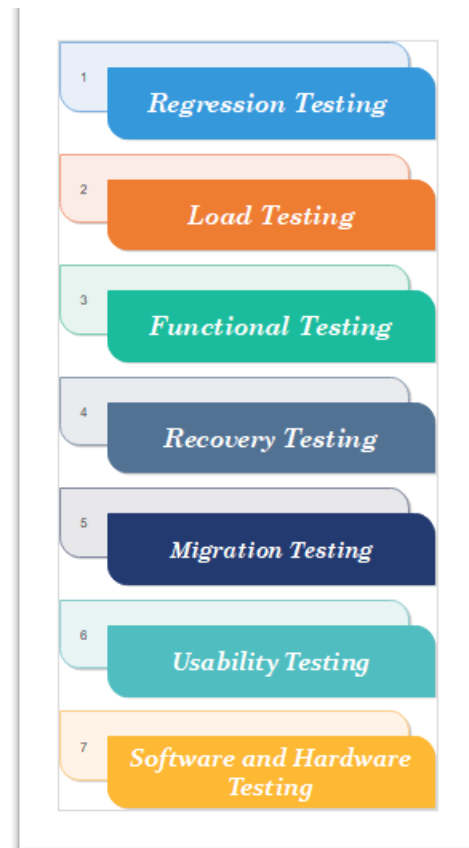
Amount

SUBMIT
CANCEL

Amazon Advertisement
Available at 10:00AM on
January 26th

Types of System Testing

System testing is divided into more than 50 types, but software testing companies typically uses some of them. These are listed below:



Acceptance Testing

The **last and fourth level** of software testing is **acceptance testing**, which is used to evaluate whether a specification or the requirements are met as per its delivery.

The software has passed through three testing levels (**Unit Testing, Integration Testing, and System Testing**). Some minor errors can still be identified when the end-user uses the system in the actual scenario.

Acceptance testing is the **squeezing of all the testing processes that are previously done**. The acceptance testing is also known as **User acceptance testing (UAT)** and is done by the customer before accepting the final product.